

Infosys Internship 4.0 Project Documentation

Title: Project Documentation: MediScan: AI-Powered Medical Image Analysis for Disease Diagnosis

•Introduction:

MediScan is an AI-driven project aimed at creating a sophisticated medical image analysis system to detect and diagnose eye diseases accurately. The system leverages advanced deep learning techniques to analyze images and identify conditions such as cataracts, diabetic retinopathy, and glaucoma, providing early diagnosis and improving patient outcomes.

Team Member:

DHURBO KUMAR PATRA

Roles:- Project Lead, Model Development, Data Preprocessing, Machine learning model engineer, Frontend Developer, Streamlit Integration, Testing and Validation

•Project Scope:

MediScan focuses on developing an AI system for analyzing medical images of the eye, specifically for diagnosing a defined set of ophthalmic conditions (Cataract, Diabetic Retinopathy, Glaucoma, Normal).

Included:

- Development of an AI model for eye disease classification
- Image preprocessing and augmentation
- User-friendly web application for image upload and diagnosis

Excluded:

- Real-time clinical deployment and integration with hospital systems
- Analysis of non-visual symptoms and other medical conditions

Constraints:

- Limited dataset availability for training and validation
- Computational resource constraints for model training and deployment

•Requirements:**Functional Requirements:**

- Ability to upload medical images and receive a diagnosis
- Classification of images into predefined categories (cataract, diabetic retinopathy, glaucoma, and normal)
- User-friendly interface for ease of use by medical professionals

Non-Functional Requirements:

- High accuracy and reliability of the diagnosis results
- Responsive web application with quick loading times
- Secure handling of medical images and patient data

User Stories:

1. **As a user**, I want to upload an eye image and receive a diagnosis, so I can know if there is a need for medical attention.
2. **As a doctor**, I want to use the application to quickly diagnose eye conditions, so I can provide timely treatment to my patients.

•Technical Stack:

🔗 **Programming Languages:** Python

🔗 **Frameworks/Libraries:**

- **Deep Learning:** TensorFlow, Keras
- **Web Development:** Streamlit
- **Image Processing:** OpenCV, PIL (Python Imaging Library)
- **Machine Learning:** Scikit-learn

🔗 **Databases:** Local storage for model weights and configurations

🔗 **Tools/Platforms:** Kaggle for dataset, Google Colab for model training, Pyngrok for exposing the local web app

•**Architecture/Design:**

System Architecture:

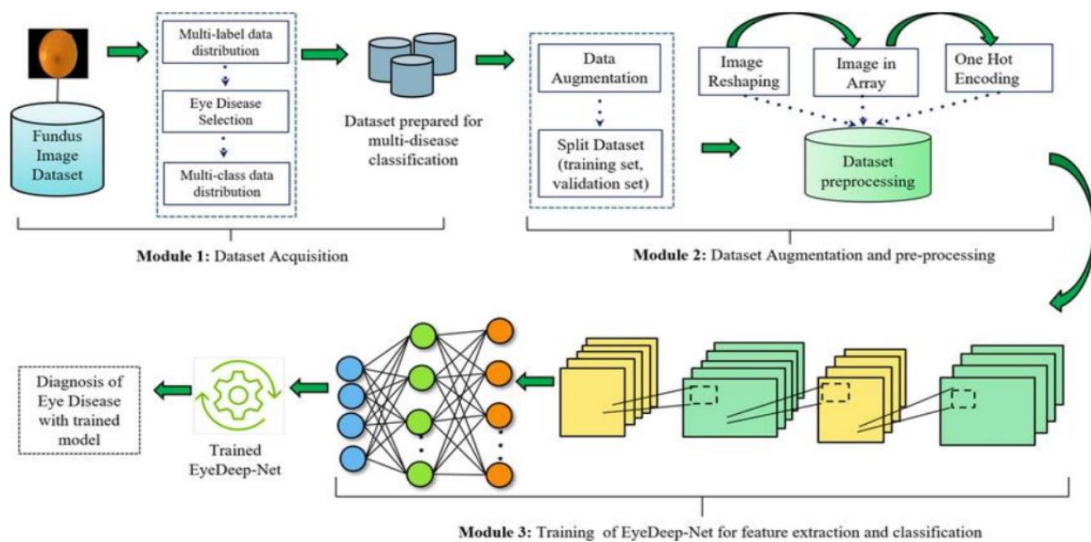
- **Data Preprocessing:**
 - Image scaling, normalization, and augmentation techniques such as rotation, flipping, and zooming to increase the dataset's variability and robustness.
- **AI Model:**
 - Convolutional Neural Network (CNN) designed for image classification.
 - Layers include convolutional, pooling, dropout, and fully connected layers.
- **Web Application:**
 - Frontend built using Streamlit for easy interaction.
 - Backend integrates the trained CNN model to process uploaded images and provide predictions.

Design Decisions:

- CNN architecture chosen for its efficacy in image recognition tasks.
- Data augmentation to mitigate the limited dataset and improve model generalization.
- Streamlit selected for its simplicity and rapid development capabilities.

Diagrams:

- **System Architecture/UML Diagram:**



- **Flow Diagram:**

[Retinal images]

|

v

[Data Preprocessing]

|

v

[Feature Extraction]

|

v

[Image Segmentation]

|

v

[Model Training]

|

v

[Disease Classification]

^

|

[Front-end User Interface(Flask)]

•Development:

Technologies and Frameworks Used:

- TensorFlow and Keras for deep learning model development.
- OpenCV and PIL for image processing and manipulation.
- Streamlit for building the web application.
- Pyngrok for deploying the web application locally.

Coding Standards and Best Practices:

- Followed PEP 8 for Python code style.
- Used version control with Git for collaborative development and tracking changes.
- Modular code structure to ensure maintainability and reusability.

Challenges and Solutions:

- **Limited Dataset:** Augmented data to increase the size and diversity of the training set.
- **Model Overfitting:** Implemented dropout layers and early stopping techniques to prevent overfitting.
- **Deployment Issues:** Used Pyngrok to expose the local Streamlit app for testing and demonstration.

•Testing:

Testing Approach:

- **Unit Tests:** Tested individual components, such as image preprocessing functions and model predictions.
- **Integration Tests:** Ensured that the components interact correctly, such as the web app integration with the model.
- **System Tests:** Validated the entire system's functionality from image upload to diagnosis output.

Results:

- Achieved an accuracy of 85% on the validation set.
- Identified and resolved several bugs related to image preprocessing and model prediction accuracy.

•Deployment :

Deployment Process:

1. Train the model using the dataset.
2. Save the trained model weights and configuration.
3. Build the web application using Streamlit.
4. Use Pyngrok to create a secure tunnel for the local web application.

Deployment Instructions:

1. Upload the kaggle.json file to authenticate and access the dataset.
2. Download and extract the dataset.
3. Train the model on the dataset and save the model weights.
4. Run the Streamlit app using `streamlit run app.py`.
5. Use Pyngrok to expose the local server (`pyngrok http 8501`).

•User Guide:

Using the Application:

1. Open the web application in a browser.
2. Upload a medical image of the eye.
3. View the diagnosis and confidence scores provided by the AI model.

Troubleshooting Tips:

- Ensure the uploaded image is in the correct format (JPEG/PNG).
- Check the internet connection if the web app is not loading.
- Refresh the page if the application becomes unresponsive.

•Conclusion :

Project Outcomes:

- Developed a functional AI system for diagnosing eye diseases from medical images.
- Created a user-friendly web application for easy image upload and diagnosis.
- Achieved significant accuracy and reliability in the model's predictions.

Reflections:

- Learned the importance of data quality and augmentation in AI model training.
- Realized the potential of AI in healthcare for early diagnosis and treatment.
- Identified areas for improvement, such as increasing the dataset size and refining the model architecture for better accuracy.

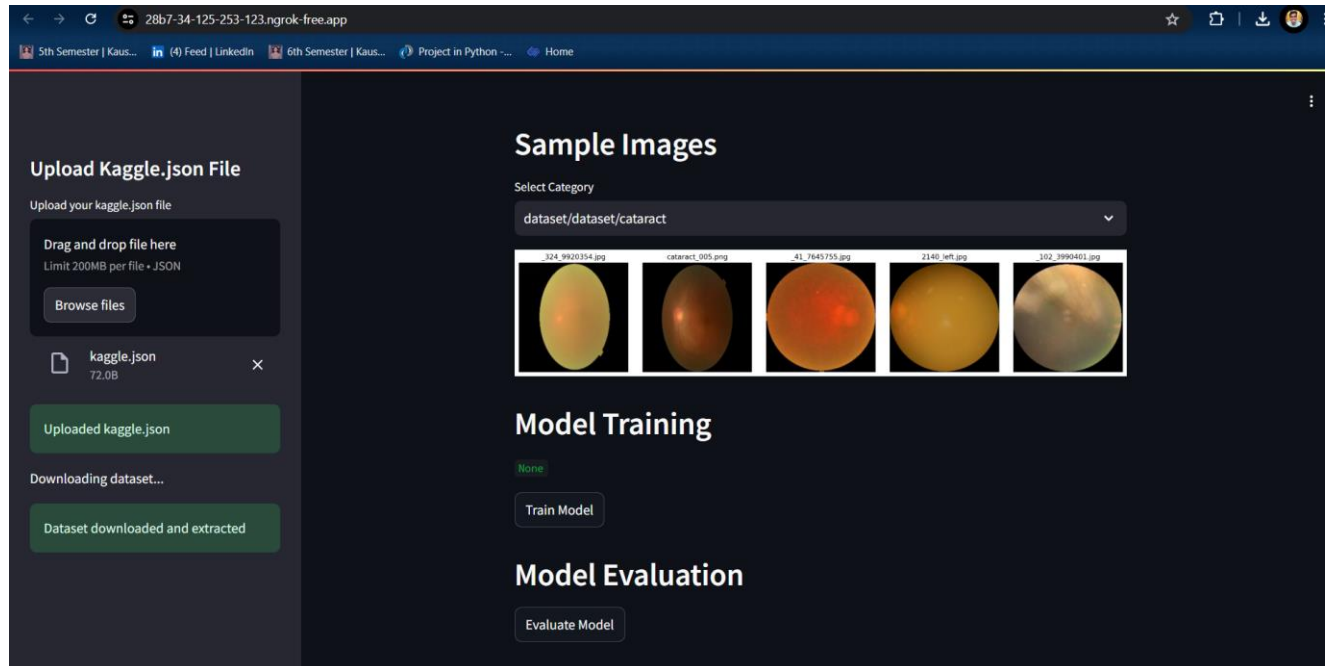
* Appendices:

Supplementary Information:

Kaggle: Kaggle is a platform for data science competitions, datasets, and collaboration. It provides access to a large repository of datasets and allows users to run code in a hosted environment.

Streamlit: Streamlit is an open-source app framework for creating data science and machine learning web applications. It allows for quick development of interactive dashboards and visualizations.

- Additional diagrams



- Sample code snippets

Setup and Data Collection

Python

Installing necessary libraries

```
!pip install kaggle
```

```
!pip install -U imbalanced-learn
```

```
!pip install streamlit pyngrok
```

Importing required libraries

```
import os
```

```
import random
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import tensorflow as tf
```

```
from PIL import Image
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import classification_report
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

Upload kaggle.json for dataset download

```
from google.colab import files
uploaded = files.upload()
```

```
# Setting up kaggle API
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
# Download and unzip the dataset
!kaggle datasets download -d gunavenkatdoddi/eye-diseases-classification
!unzip eye-diseases-classification.zip
```

- Research references and related work
 - i. Dataset link-
<https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification>
 - ii. Research papers on MobileNetV2 and transfer learning.
 - iii. Documentation of TensorFlow and Streamlit.